Weather Data Analysis Using Python

Final Project Report

Author: Pranita Shirsekar
Tools: Python, Pandas, NumPy, Matplotlib, Google Colab
Project type: Exploratory Data Analysis (EDA)

1. Executive Summary

This project explores weather observations from multiple locations using Python. The workflow covers data inspection and cleaning, date-time feature engineering, descriptive statistics, monthly and seasonal aggregation, visualizations, and percentile-based identification of unusual temperature and precipitation observations.

The working dataset contains six selected columns: last_updated, country, location_name, temperature_celsius, humidity, and precip_mm. The observed date range is 16 May 2024 to 11 October 2026, with 878 unique calendar dates and 268 unique locations. Combined summaries describe the overall dataset rather than one city's climate.

2. Project Objectives

• Inspect the structure and quality of the weather dataset.

• Convert timestamps to datetime and extract year, month, month name, and season.

• Calculate mean, median, and standard deviation with NumPy.

• Compare monthly and seasonal weather metrics using Pandas.

• Create temperature and precipitation visualizations with Matplotlib.

• Identify percentile-based extreme temperature and high-precipitation observations.

• Summarize results while acknowledging data limitations.

3. Dataset Overview

Item	Details
Original row count observed	170,538
Selected columns	6
Unique locations	268
Unique calendar dates	878
Earliest timestamp	2024-05-16 01:45:00
Latest timestamp	2026-10-11 18:30:00
Exact duplicate rows before deduplication	1
Missing values in selected columns at initial inspection	0

Selected columns: last_updated (timestamp), country, location_name, temperature_celsius (°C), humidity (%), and precip_mm (mm). The original dataset had additional fields; six relevant columns were retained for focused analysis.

4. Data Preparation and Quality Checks

• Loaded and inspected the dataset with Pandas.

• Selected six relevant columns.

• Checked data types, missing values, and duplicate rows.

• Detected one exact duplicate row in the initial selected dataset and removed duplicates in the cleaning workflow.

• Converted last_updated to datetime.

• Extracted year, month, and month_name, and created a season feature.

The selected columns initially had no missing values. Invalid timestamps should also be checked after datetime conversion, because invalid values converted with errors='coerce' become missing datetimes.

5. Analysis Performed

5.1 Descriptive statistics with NumPy

Mean, median, and standard deviation were calculated for temperature, humidity, and precipitation. The exact calculated values are not stated here because the notebook's results table was not available for verification.

5.2 Monthly analysis

The data was grouped by calendar month to calculate average temperature, average humidity, total precipitation, and average precipitation per observation. Monthly total precipitation combines observations across all years and locations; it is not typical rainfall for one city in one year.

5.3 Seasonal analysis

Records were grouped into Winter (December–February), Summer (March–May), Monsoon (June–September), and Post-monsoon (October–November). These categories follow a Mumbai/India calendar and should not be treated as locally accurate seasons for all 268 locations.

5.4 Visualizations

• Line plot: monthly average temperature over time, aggregated across all locations.

• Bar chart: total precipitation by calendar month across the full dataset.

• Histogram: distribution of temperature observations.

• Bar chart: average temperature by the assigned season.

The line chart is a combined multi-location series, not the historical temperature trend of one city.

5.5 Extreme observations

Temperature observations at or below the 5th percentile or at or above the 95th percentile were flagged. High-precipitation observations were identified at or above the 95th percentile among records with precipitation greater than zero. The warmest, coolest, and highest-precipitation records were also identified. These are dataset-relative thresholds, not official weather-warning criteria.

6. Key Observations Supported by Available Outputs

• The observed date range is 16 May 2024 through 11 October 2026.

• The dataset includes 878 unique calendar dates and 268 unique locations.

• Several locations have records on nearly all observed calendar dates, although exact continuity should be verified before treating the dataset as a complete daily time series.

• One exact duplicate row was detected before deduplication.

• No missing values were reported in the six selected columns during initial inspection.

The numerical outputs for descriptive statistics, monthly and seasonal summaries, charts, and extreme records were not available for independent verification. Therefore, no specific warmest month, coolest month, wettest season, average temperature, or extreme-observation count is claimed in this report.

7. Limitations and Considerations

• Combined multi-location averages can hide local differences and should not be attributed to a particular city.

• Observation counts and timestamps may vary by location; coverage should be checked before making time-series claims.

• The selected seasonal categories follow the Mumbai/India calendar and are not geographically appropriate for every location.

• Total precipitation summed across years and locations is cumulative, not a typical monthly rainfall value.

• Percentile-based extremes are dataset-relative and do not replace official meteorological warnings.

• Final numerical claims should be checked against the actual notebook tables and plots.

8. Conclusion

This project demonstrates an exploratory data analysis workflow using Python. Pandas supports inspection, cleaning, transformation, and aggregation; NumPy supports descriptive statistics and percentile thresholds; and Matplotlib supports visualization of temperature and precipitation patterns.

The dataset's breadth—268 locations and 878 distinct calendar dates—enables broad comparisons but requires careful interpretation. Reliable conclusions should match the notebook outputs and clearly state whether they describe the combined dataset or a single location. Verify the output tables and charts before adding specific numerical findings.

9. Reproducibility

• Open the project's .ipynb notebook in Google Colab or Jupyter Notebook.

• Load the source weather dataset.

• Run notebook cells in order.

• Review statistics tables, monthly and seasonal summaries, charts, and extreme-observation outputs.

• Update the report with verified numerical findings if outputs change.

Skills demonstrated: Python, Pandas, NumPy, Matplotlib, data cleaning, feature engineering, descriptive statistics, groupby aggregation, exploratory data analysis, data visualization, and interpretation of analytical limitations.